Server-Side Key Management

Trusted Server Model for Zero-Effort File Sharing

CloudVault - Secure Cloud Storage System

Hybrid Encryption: AES-256-GCM + RSA-2048

The cleanest solution where the owner does absolutely NOTHING extra after upload.

1. The Key Idea

At upload, encrypt the AES key TWICE - once for the owner, and once for the server itself. Now the server can handle all sharing automatically. The owner just clicks "Share" - zero crypto work on their end.

Upload (one-time, by owner)

1. Generate random AES-256 session key
2. Encrypt file data with AES-256-GCM
3. Encrypt AES key with owner's RSA public key -> for owner's access
4. Encrypt AES key with server's RSA public key -> for server to manage sharing
5. Upload encrypted file + both encrypted keys to server

Sharing (owner just clicks "Share with User B" - that's it)

1. Server decrypts AES key using server's RSA private key
2. Server encrypts AES key with recipient's RSA public key
3. Server stores the new encrypted key entry for the recipient
4. Owner did nothing. Just one click.

Download (by any authorized user)

1. Server sends user their encrypted AES key + encrypted file
2. User decrypts AES key with their RSA private key
3. User decrypts file with AES key -> Original File

2. Owner's Experience

The owner never performs any manual cryptographic operations after the initial upload:

Action	What Owner Does	Crypto Work by Owner
Upload	Selects file, clicks Upload	Automatic (background)
Share	Selects recipient, clicks Share	None. Zero.
Revoke	Clicks Revoke	None. Server deletes key

3. Complete Flow Diagram

UPLOAD

```
Owner --> AES_Key --> Encrypt File
                |-- Encrypt with Owner's RSA Pub  --> stored (owner access)
                |-- Encrypt with Server's RSA Pub --> stored (sharing use)
```

SHARE (owner clicks one button)

```
Server --> Decrypt AES_Key with Server's RSA Priv
       --> Re-encrypt with Recipient's RSA Pub
       --> Store for recipient
       (Owner does NOTHING)
```

DOWNLOAD

```
Recipient --> Gets their RSA-encrypted AES Key
          --> Decrypts with their RSA Private Key
          --> Decrypts file with AES Key --> Original File
```

4. Why This Works Within the Project Scope

Project requirement: "secure cloud storage system using hybrid encryption (AES + RSA)... secure key management... controlled file sharing... protect from unauthorized access"

- AES + RSA hybrid - AES encrypts file data, RSA encrypts keys. Fully preserved.
- Secure key management - Server manages encrypted keys using its own RSA key pair. The AES key is NEVER stored in plaintext - it is always RSA-encrypted.
- Controlled file sharing - Owner controls WHO gets access. Server handles HOW.
- No user private key is ever shared - Each user still decrypts with their own RSA private key.
- This is exactly how real cloud systems work (AWS KMS, Google Cloud KMS, Azure Key Vault all use a trusted server key management model).

5. Security Analysis

Concern	Answer
Server sees AES key?	Yes - server is a trusted entity (standard model)
External attackers?	Protected. File=AES encrypted, keys=RSA encrypted
User private keys shared?	NEVER. Keys stay in browser (IndexedDB)
Unauthorized access?	No. Only users with a key entry can decrypt
Key stored in plaintext?	No. AES key is always RSA-wrapped

6. Implementation Architecture

Key Envelope Table (Database)

Each file has multiple key entries stored in the FileKeyEntry table. Each entry contains the AES session key encrypted with a different RSA public key:

Key Type	Encrypted With	Purpose
OWNER	Owner's RSA Public Key	Owner can download/decrypt
SERVER	Server's RSA Public Key	Server can re-encrypt for sharing
SHARED	Recipient's RSA Public Key	Recipient can download/decrypt

Technology Stack

Layer	Technology
Backend	Java 17, Spring Boot 3, Spring Security, JPA
Database	H2 (file-based, embedded)
Encryption	JCA: AES-256-GCM, RSA-2048-OAEP (SHA-256)
Frontend	React 19, Vite, Axios
Client Crypto	Web Crypto API (RSA keygen, AES decrypt)
Auth	JWT (JSON Web Tokens) + BCrypt

API Endpoints

Endpoint	Description
POST /api/auth/register	Register user (sends RSA pub key)
POST /api/auth/login	Login (returns JWT token)
POST /api/files/upload	Upload and encrypt file
GET /api/files/download/{id}	Download encrypted file
POST /api/share	Share file with another user
DELETE /api/share/{fId}/{user}	Revoke access from a user

7. Why Does the Server Store the Recipient's Key?

A common question: "Why does the server store a key entry for the recipient?" The answer: it is NOT storing the recipient's private key or public key. It stores the FILE'S AES key, encrypted FOR the recipient.

What is Actually Stored (Key Envelope Table)

Each file has multiple rows in the Key Envelope Table. Each row holds the same AES session key, but encrypted with a DIFFERENT user's RSA public key:

File ID	User	What is Stored
file_001	owner	AES key encrypted with OWNER's RSA public key
file_001	server	AES key encrypted with SERVER's RSA public key
file_001	userB	AES key encrypted with USERB's RSA public key

Why Store It?

Because User B will download the file LATER - maybe tomorrow, maybe next week. When they do:

1. Server looks up: "Does User B have a key entry for this file?"
2. YES -> sends them the encrypted AES key + encrypted file
3. User B decrypts with their RSA private key -> gets original file

If the server did NOT store it, it would have to re-encrypt every single time User B downloads. That is wasteful. Store once -> download anytime.

What is NOT Stored

- Nobody's RSA PRIVATE key - never leaves the user's browser
- The AES key in PLAINTEXT - it is always RSA-encrypted
- Only the AES key encrypted with that specific user's RSA public key (~256 bytes)

Revoke = Just Delete That Row

When the owner clicks "Revoke" for User B, the server simply DELETES that row from the Key Envelope Table. Now User B has no key entry, cannot decrypt, and has no access. The encrypted file is useless without the matching key entry.

So it is essentially an ACCESS CONTROL LIST, but cryptographically enforced. Without the key entry, the encrypted file is meaningless to anyone.

8. Conclusion

The Server-Side Key Management (Trusted Server Model) is the simplest, most practical, and zero-effort solution for the file owner. It is fully within the project's scope of AES + RSA hybrid encryption.

Key takeaways:

- The file is uploaded and encrypted ONCE. No re-uploads for sharing.
- The owner clicks one button to share. The server handles all re-encryption.
- Each user decrypts with their own RSA private key. No keys are ever shared.
- Revoking access is as simple as deleting the recipient's key entry.
- This model is used by AWS KMS, Google Cloud KMS, and Azure Key Vault in production.

This is the best approach that satisfies all project requirements
while providing the best possible user experience for file owners.